

Software Requirement Engineering

Difference b/w Software Engineering & SRE

SE (Software Engineering)

1. Focuses on designing, developing, testing and maintaining software system.
2. Concerned with entire SDLC.
3. Deals with "how" of software development.
4. Involves programming, coding and software construction.
5. Aims to produce a working software product.

SRE (Software Requirement Engineering)

1. Focuses on defining, analyzing, documenting and maintaining software requirements.
2. Concerned with understanding the needs of stakeholder and users.
3. Involves requirement gathering, analysis, specification and validation.
4. Deals with the "what" of software development.
5. Aims to produce a clear, complete and unambiguous (ضوح) set of requirements.

Classification of requirements:

Functional Requirements:

1. Describe what the system must do.
2. Define the task and functions the system must perform.
3. Examples: login functionality, data processing, user authentication.

Non-functional Requirements:

1. Define the constraints and quality attributes of the system.
2. Describe how the system should perform.
3. Example: Performance, scalability, security, usability.

Business Requirements:

1. Define the business goals & objectives.
2. Describe the needs and constraints of the organization.
3. Examples: Increase sales, reduce costs, improve customer satisfaction.

User Requirements:

1. Describe the needs and expectations of the end-users.

2. Define the tasks and functions

users must perform.

3. E.g. user interface, user experience workflow.

System Requirements:

1. Describe the technical requirements

of the system

2. Define the hardware, software and network infrastructure needed.

3. E.g. operating system, database network protocols.

Domain Requirements:

1. Describe the specific rules & regulations of the domain

2. Define the industry-specific standards and best practices.

3. Examples: healthcare regulations, financial industry standards.

Quality Requirements:

1. Describe the levels of quality and performance expected

2. Define the metrics & benchmarks for evaluation

3. E.g. response time, throughput, error rate.

Requirement Process

It is a systematic approach used in SE to Elicit, analyze, document, validate and manage requirements for a software system.

1) Requirement Gathering:

1. Identify stakeholder and users need
2. Collect requirements through interviews, surveys and observations.
3. Define the scope and boundaries of the project.

Requirement Analysis:

1. Examine and Interpret (know) requirements.
2. Identify conflicts, ambiguities (unclear) and inconsistencies
3. Define the functional and non-functional requirements

Requirements Specification:

1. Document The requirements in a clear and concise manner.
2. Use natural language, diagram and model to describe the requirements.
3. Produce a requirements specification document.

Requirements Verification:

Check the requirements for completeness,

1. Check the requirements for accuracy and consistency
2. Ensure that requirements are unambiguous and measurable
3. Verify that the requirement meet the stakeholder needs.

Requirement Validation:

1. Ensure that the requirements are correct and meet the user needs
2. Validate requirements through prototyping, testing & feedback.
3. Confirm that requirements are feasible and achievable.

Requirement Management:

1. Track and manage changes to the requirements
2. Ensure that the requirements are up-to-date and relevant
3. Maintain a requirements traceability matrix.

Requirement Communication:

1. Communicate the requirements to Stakeholders and team members
2. Ensure the everyone understands the requirements
3. Use appropriate communication channels and techniques

SRE

Levels/Layers of Requirements

1) Business requirement (Why) (of stakeholders)
 → User software (How) (Expectation)

→ Functional (What) (Feedback)

BR (User)	UoR (User)	S.R. (Version)
Outline measurable goals & objectives	Reflect specific needs & expectation of users from the system	Identify FRs, NFRs & use case
Define why making project	Describe who & project (How used & which people are used)	Describe how a product is used
Match project goals with stakeholder goals	Describe how users can interact with it	Describe product (software) as functional & non-functional (Babbar team) & deliverable
Maintain BRD with requirements, update arrange	create URs and make them them part of BRD	compose SRs & optionally FPS

Requirement Characteristics:

Clear:

- Easy to understand
- Unambiguous (2%)
- Free from technical terminologies

Concise:

- Brief and to the point
- Avoid unnecessary details
- Focus on essential information

Complete:

- Covers all necessary information
- Includes all relevant details
- Leaves no room for misinterpretation

Consistent

- Aligns with other requirements
- Uses consistent terminology
- Avoids contradictions

Feasible

- Realistic and achievable
- Takes into account technical and resource constraints
- Can be implemented within budget and timeline.

Measurable

- Quantifiable and verifiable
(precise way to measure performance)
- include clear metrics and bench marks (reference point to evaluate/improve performance)
- Allows for objective evaluation

Traceable

- Links to business goals and objectives

- Connected to user needs and expectations

- Easy to track and monitor

Verifiable

- Can be tested and validated
- include clear acceptance criteria
- Ensures requirement is met

Analyzing Quality Requirements

Quality requirements also known as non-functional requirements or quality attributes, specify the overall quality characteristics that a system must perform.

1. Performance:

Define how well the system must perform under certain conditions

- Response Time
- Throughput
- Scalability

2. Security:

Address the protection of data, resources and functionality from unauthorized access

- Authentication
- Authorization
- Data protection

3. Usability:

Focuses on user experience and ease of use of the system.

- User experience
- Accessibility
- Error handling

4. Maintainability:

Specify how easily the system can be modified, extended or updated over time

- Modularity
- Scalability
- Upgradability

5. Portability:

System can be deployed and run on different hardware, OS & environment.

- Compatibility (ability to operate with other system)
- Adaptability
- Interoperability (communication with 3rd party)

6) Reliability:

specify the system's ability to perform consistently and reliably over time without failure.

- Uptime
- Downtime
- Failure rate

Requirements Evolution:

refers to the process of changing, updating and refining software requirements over time to ensure they remain relevant, accurate and aligned with the changing needs of stakeholders and the system.

Why Evolve Software Requirements?

- Changing stakeholders needs & expectations
- New functional & non-functional requirements
- Emerging technologies and innovations
- Changing business goals & objectives
- Ensure software remains relevant
- Addressing defects, error
- Improving software quality and performance

Software Requirements Evolution Team:

- Requirements Engineers
- Project Managers
- Stakeholders
- Product Owners
- Change Management Team
- Subject Matter Experts (SMEs)
- Developers and Architects
- Tester & Quality Assurance Team
- Business Analysts

- Tools for Software Requirements Evolution:
- Requirements Management Tools (RMTs)
DOORS, TestLink, Jama Connect
 - Version Control Systems (VCSs)
Git, SVN, Mercurial
 - Change Management Systems (CMSs)
JIRA, ServiceNow, BMC Helix
 - Agile Project Management Tools:
Asana, Microsoft Azure DevOps
 - Requirements Prioritization Tools:
MOSCOW, Kano Model, Pareto Analysis
 - Collaboration & Communication Tools
Stack, Microsoft Teams, Email

Requirement Evolution Activities:

- Requirement ~~collection~~ gathering and analysis
- Impact analysis (assessing the effects of changes)
- Requirement updating and refinement
- Verification & Validation of updated requirements
- Communication & stakeholder management

Advantage:

1 - Improved Quality

2 - Reduced Risk:

3 - Flexibility

4 - User Satisfaction

5 - Business Alignment

(Ensure software align with changing business needs & objectives)

Disadvantage:

1 - Increased Complexity

2 - Cost & Resource

3 - Stakeholder Management

4 - Documentation & Traceability (Time consuming)

5 - Testing & Validation (Time & Cost)

Requirement Prioritization:

It is the process of ranking requirements in order of importance and urgency (value) to ensure the most critical requirements are addressed first.

Goals :-

- Ensure that most valuable requirements are addressed first.
- Reduce project Risk (minimize the risk of wasting resources)
- Optimize resource allocation
- Increase project success
- Improve stakeholder satisfaction
- Supports Agile Development: (allowing for flexibility & adaptability)

Factors to consider:

1. Business value:

How aligned is the requirement with business goals and objectives.

2. Customer needs:

How well requirement meet customer expectations and needs.

Risk:

What is potential impact and likelihood of negative consequences if the req. is not met.

Complexity:

How difficult is req. to implement and what resources are required.

Cost:

What is estimated cost of implementing the req.

Urgency: (time)

How time-sensitive is the requirement and what are the consequences of delaying it.

Dependencies:

Are there any interdependent req. that need to be considered.

Stakeholder input:

What are the priorities of stakeholders and has there any kind of conflict. All

Team agree on same point.

Technique/Method for Requirement Prioritization

1. MoSCoW Method:

Must have:

Critical, cannot be skipped

Should have:

Important, but can be skipped if necessary

Could have:

Nice to have but not essential

Won't have:

Not necessary, can be skipped

2. Kano Model:

Represent requirements on a graph with two axes

Satisfaction:

How much will the requirement satisfy stakeholders?

Effort: How much ~~time~~ effort

is required to implement the requirement.

Requirements fall into three categories:

Must be: High satisfaction, low effort

More is better:

High satisfaction, high effort

Delighter: Low satisfaction, low effort.

Pareto Analysis:

Identify the 20% of requirements that deliver 80% of the value.

Prioritization Matrix:

Create a table with requirements on one axis & prioritization factor (e.g. business value, customer needs, risk)

Score each req. against each factor and total score determine priority.

Best Practices

- Involve stakeholder in prioritization process
- Use a consistent prioritization method
- Prioritize requirement regularly
- Consider both short-term & long-term goals
- Be flexible & adapt to changing priorities

Common Pitfalls:

- Prioritizing based on person opinions rather than objective criteria
- Failing to consider dependencies and constraints
- Not involving stakeholder in prioritization process
- Prioritize too many req. as high priority
- Not regularly reviewing & updating priorities.

Requirement Traceability

Traceability is a combination of two words. Trace means to find something/someone and ability means capability/skill to do something. So,

Requirement Traceability is the ability to ^{track & document} trace a requirement ^{origin} forwards and ^{implementation} backwards in the development lifecycle.

Goals

- Provide better quality of product
- Find any risk → Easier impact analysis
- Keep & verify the record of history and production of an item (documentation)

Types of RT:

① Forward Traceability: (origin)
tracks requirement from the beginning to the end (implementation)

② Backward Traceability:
trace how project element relate to initial requirements. trace requirement from their implementation back to their

origin. It helps in understanding why certain decisions were made during implementation phase by linking them back to the initial requirements.

Bidirectional traceability:

Ability to trace and link requirements both forward and backward. We can track how each requirement is connected to other requirements ensuring that changes made in one area are reflected in all related areas. It helps maintain consistency and ensure that all project elements are aligned.

Requirement traceability matrix

RTM is a way to document the traceability of req. It is a spreadsheet that tracks req. through the development process, collecting data about the req. related coding the testing has been done and the original reason to include the req. and in the same place.

RTR (Req. Traceability Report)

Tradeoff Analysis

It is a systematic process of evaluating & prioritizing conflicting requirements/constraints or options by considering their relative cost, benefits & risk.

Goals :-

- Identify & evaluate trade-offs
- Prioritize requirements/options
- Make informed decisions
- Optimize resource allocation
- Minimize risks & maximize benefits

Steps

1. Identify trade-offs (conflict req.)
2. Define evaluation criteria:
Evaluate the potential options and their impact on the project objectives.
3. Analyze cost & Benefits:
Analyze the cost & benefits of each options based on the potential impact on the project objectives.

4. Prioritize options

Prioritize options based on their potential impact on project objectives

5. Select Best options!

6. Validate the Decisions.

Validate the decision with stakeholders and ensure that it meets their needs & expectations.

Techniques:-

1. Cost - Benefit Analysis

Evaluates the potential costs & benefits of each option.

2. Pareto Analysis:

Identifies the most significant requirements.

3. Sensitivity Analysis:

Analyse the impact of changing requirements / constraints

4. SWOT Analysis:

Focus on strength, weakness, opportunity occurrence of requirements

Best Practices:

- Involve Stakeholders in the Process
- Consider multiple Perspectives.
- Use Quantifiable data
- Document the Process
- Review & update trade-off analysis regularly

Pitfalls:

- Failing to consider multiple options
- Ignoring risk
- Not involving stakeholders
- Not regularly reviewing & updating trade-off analysis

SRE

Risk

Risk is an uncertain event or condition that if occurs has positive or negative effect / impact on a project.

Risk management

It is the process of identifying, assessing and mitigating risks to minimize their impact on project / organization.

• Proactive risk management

It involves identifying and mitigating potential risks before the occur. often through strategies like risk assessment, planning and prevention measures.
→ minimizing likelihood & impact of risk.

• Reactive risk management

It deal with responding to risk after they have happened, typically through strategies like damage control & recovery efforts.

→ focus on minimizing damage and restoring ^{normal} operations after a risk event.

Risk analysis

It is a component of risk management.

Risk analysis involves identifying and assessing risk by analyzing their likelihood and potential impact.

Component

1. Risk Identification:

Identifying potential risk that could affect the project. This involves brainstorming sessions, interviews with stakeholders, reviewing historical data and using various techniques such as checklist, flowchart to identify both internal and external risks.

2. Risk Assessment:

Once risks are identified, they need to be assessed to understand their likelihood and potential impact. This involves analyzing factors such as probability of occurrence and timeframe in which they could occur.

Risk assessment often utilizes qualitative or quantitative methods such as probability distribution or statistical model to evaluate risks and prioritize them based on their significance.

3. Risk Evaluation:

It is process of figuring out how serious a risk is and how it could affect our plan. It involves looking at different factor such as how risk is happen and how much it cause damage if occur. e.g. Planning outdoor event, rain came. Risk evaluation helps us to understand risk so we can decide how to with it.

4. Risk Mitigation:

Risk mitigation involve steps to reduce the impact of risk. It could involve avoiding the risk, transferring it to another party, reducing its likelihood or having a contingency plan.

Contingency Planning

It involves creating backup plan if original plan does not work out. Be ready for unexpected issues and ready to deal with it.

4. Risk Monitoring & Reviews:

Risk analysis is an ongoing process. It is important to continuously maintain the identified risks, assessing new ones that may occur and review the effectiveness of the mitigation strategies in place.

Techniques to deal with risk

Brainstorming:

Team members gather to generate ideas and identify potential risk related to software requirements.

Participants freely express their thoughts on possible risks without judgment, encouraging creativity and diverse perspectives.

Interviews:

Stakeholders including users, developers and project managers are individually interviewed to gather insights (info. collection) on potential risk.

Open-ended questions help elicit (extract) information about concerns, expectations and uncertainties related to requirements.

Checklists:

Pre-defined lists of common risks and issues are used to systematically review requirements.

Team members go through a checklist, marking off potential risks that may apply to the project.

SWOT analysis of project

Description: Examines internal and external factors influencing Project success or failure.
How it works: - Identifies strengths and weaknesses within the project team and opportunities and threats from the external environment.

Risk workshops

Involves a structured, facilitated session where team members collectively identify and assess risks.

How it works: Facilitator guide the team through discussions, ensuring active participation and collaboration in risk identification and assessment.

Use case analysis:

Focuses on analysing use cases to identify potential risks associated with functional requirements.

How it works: Examine how user interact with the system, helping identify potential pitfalls or misunderstandings in requirements.

Requirement Management:

The process of defining, gathering, analyzing, documenting, prioritizing and tracking requirements throughout the project lifecycle.

Goals:

- Ensure req. are clear, complete, unambiguous
- Ensure req. are aligned with stakeholder goals & needs.
- Reduce error, defects
- Improve communication & collaboration among stakeholders

Activities

1. Requirements gathering/Elicitation
2. Requirements analysis and validation
3. Requirements specification & documentation
4. Requirements verification & testing
5. Requirements traceability & prioritization
6. Requirements Change Management

Best Practices:

- Use a req. management tool (RMT)
- Involve stakeholder throughout the process
- Prioritize requirements
- Use traceability & impact analysis
- Review & update req. regularly

Benefits:

- Improved project success
- Reduced project risk
- Increased stakeholder satisfaction
- Better communication & collaboration
- Improved compliance & reduce errors

Challenges:

- Managing changing requirements
- Ensuring stakeholder agreement
- Dealing with conflicting requirements
- Maintaining traceability & impact analysis

Interaction b/w Requirements and Architecture

It is fundamental for creating a system that fulfil the stakeholder needs while being technically feasible and maintainable.

Requirements:

- Define what the system must do (functional) and how it should perform (non-functional)
- Describe user's needs & expectations

Architecture:

- Describe how a system will be organized and how it will behave in order to fulfil these requirements.

Interaction:

- Requirements inform & shape the architecture
- Architecture helps to clarify & refine requirements
- Requirements are used to validate and test the architecture.
- Architecture provides a framework for implementing requirements.

Steps:

- Understanding requirements
- Translating requirements into architecture
- Architectural design decision
- Iterative refinement
- Verification and Validation
- Traceability & Management

Importance:

- Aligned with stakeholder needs
- Quality Assurance
- Risk Mitigation
- Cost and Time Efficiency
- Scalability & Flexibility
- Communication & Collaboration
- Traceability & Compliance
- Iterative development